

```

/**
 * =====
 * EchoknightAI – Webhook Server
 * Stack : Node.js + Express
 * Purpose: Receives a POST request containing an X (Twitter)
 *           post, analyses it with Claude, and returns a
 *           polished DRAFT reply for human review & manual posting.
 * =====
 *
 * SETUP INSTRUCTIONS
 * -----
 * 1. Install dependencies:
 *     npm install express dotenv
 *
 * 2. Create a .env file in the same directory:
 *     ANTHROPIC_API_KEY=your_anthropic_api_key_here
 *     PORT=3000
 *     WEBHOOK_SECRET=your_secret_token    # optional but recommended
 *
 * 3. Run the server:
 *     node echoknight-webhook.js
 *
 * 4. Expose it publicly (for Zapier / Make / etc.):
 *     npx ngrok http 3000
 *     – or deploy to Railway / Render / Fly.io
 *
 * WEBHOOK ENDPOINT
 * -----
 * POST /webhook/echoknight
 *
 * REQUEST BODY (JSON):
 * {
 *   "post_text"      : "The post content you want to reply to",
 *   "post_author"    : "AuthorDisplayName",           // optional
 *   "post_url"       : "https://x.com/...",          // optional
 *   "context"        : "Any extra context you want"  // optional
 * }
 *
 * RESPONSE (JSON):
 * {
 *   "status"         : "success",
 *   "tone_detected"  : "Witty/Funny",
 *   "draft_reply"    : "...",
 *   "label"          : "[DRAFT FOR YOUR REVIEW]",

```

```

*      "reminder"      : "Review and manually post this reply on X."
*    }
*    =====
*/

require("dotenv").config();
const express = require("express");
const app = express();
app.use(express.json());

const PORT = process.env.PORT || 3000;
const ANTHROPIC_API_KEY = process.env.ANTHROPIC_API_KEY;
const WEBHOOK_SECRET = process.env.WEBHOOK_SECRET; // optional guard

// — Optional: simple secret-token guard —————
function verifySecret(req, res, next) {
  if (!WEBHOOK_SECRET) return next(); // skip if not configured
  const token = req.headers["x-webhook-secret"];
  if (token !== WEBHOOK_SECRET) {
    return res.status(401).json({ error: "Unauthorized: invalid webhook secret." });
  }
  next();
}

// — EchoknightAI system prompt —————
const SYSTEM_PROMPT = `
You are EchoknightAI, an AI assistant that crafts 10x more perceptive,
personally-attuned, and genuinely valuable draft replies to X (Twitter) posts.

CORE TASK
-----
You will receive an X post. Your job is to:
1. Analyse its explicit topic AND the deeper underlying message or emotion.
2. Detect the author's tone and communication style precisely.
3. Generate ONE polished draft reply that a human can review and manually post.

ANALYSIS LAYERS
-----
Layer 1 - Post Analysis
  • What is the explicit topic?
  • What is the underlying emotion, challenge, or unexpressed question?
  • What would a truly valuable reply address?

Layer 2 - Tone Detection
  • Study word choice, punctuation, energy, and mood signals.
  • Classify into ONE primary tone mode:
    🎩 Formal      — Sharp, composed, intelligent

```

- 😊 Witty/Funny – Clever, light, culturally aware
- 💙 Emotional – Empathetic, sincere, warm
- 🌹 Philosophical – Reflective, layered, poetic

Layer 3 – Draft Generation

- Mirror and match the detected tone exactly.
- Open with a natural hook – never generic openers like "Great post!".
- Add a perspective, insight, or value that enriches the conversation.
- Keep it concise and platform-appropriate (under 280 characters preferred, max 500 if depth is warranted).
- Sound like a thoughtful, articulate human – never robotic.

OUTPUT FORMAT (strict JSON, no markdown fences)

```
-----
{
  "tone_detected": "<one of the four tone labels above>",
  "draft_reply": "<the draft reply text only – no labels inside the reply itself>"
}
```

COMPLIANCE RULES (non-negotiable)

- ```

```
- Output is always a DRAFT for human review – never instruct auto-posting.
  - No harassment, hate speech, spam, or platform violations.
  - Treat every author and their audience with full dignity and respect.
  - Never fabricate facts or make claims you cannot support from the post itself.
- ```
`;
```

```
// — Main webhook handler —————
app.post("/webhook/echoknight", verifySecret, async (req, res) => {
  const { post_text, post_author, post_url, context } = req.body;

  // Basic validation
  if (!post_text || typeof post_text !== "string" || post_text.trim() === "") {
    return res.status(400).json({
      error: "Missing required field: post_text must be a non-empty string.",
    });
  }

  if (!ANTHROPIC_API_KEY) {
    return res.status(500).json({
      error: "Server misconfiguration: ANTHROPIC_API_KEY is not set.",
    });
  }

  // Build the user message
  let userMessage = `POST TO REPLY TO:\n"${post_text.trim()}"`;
  if (post_author) userMessage += `\n\nAUTHOR: ${post_author}`;
```

```

if (post_url)    userMessage += ` \nURL: ${post_url}`;
if (context)    userMessage += ` \n\nEXTRA CONTEXT: ${context}`;
userMessage += "\n\nAnalyse this post and return your JSON response.";

try {
  // Call Anthropic Claude API
  const anthropicRes = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-api-key": ANTHROPIC_API_KEY,
      "anthropic-version": "2023-06-01",
    },
    body: JSON.stringify({
      model: "claude-sonnet-4-20250514",
      max_tokens: 1000,
      system: SYSTEM_PROMPT,
      messages: [{ role: "user", content: userMessage }],
    }),
  });

  if (!anthropicRes.ok) {
    const errBody = await anthropicRes.text();
    console.error("Anthropic API error:", errBody);
    return res.status(502).json({ error: "Upstream AI error. Please try again."
  }

  const data = await anthropicRes.json();
  const rawText = data.content
    .filter((b) => b.type === "text")
    .map((b) => b.text)
    .join("");

  // Safely parse JSON from Claude's response
  let parsed;
  try {
    const clean = rawText.replace(/```json|```/g, "").trim();
    parsed = JSON.parse(clean);
  } catch {
    // Fallback: return raw text if JSON parsing fails
    return res.status(200).json({
      status: "success",
      tone_detected: "Unknown",
      draft_reply: rawText,
      label: "[DRAFT FOR YOUR REVIEW]",
      reminder: "Review and manually post this reply on X.",
    });
  }
}

```

```

    }

    return res.status(200).json({
      status: "success",
      tone_detected: parsed.tone_detected || "Unknown",
      draft_reply: parsed.draft_reply || "",
      label: "[DRAFT FOR YOUR REVIEW]",
      reminder: "Review and manually post this reply on X.",
    });

  } catch (err) {
    console.error("Server error:", err);
    return res.status(500).json({ error: "Internal server error. Check logs." });
  }
});

// — Health check —————
app.get("/", (req, res) => {
  res.json({
    service: "EchoknightAI Webhook",
    status: "running",
    endpoint: "POST /webhook/echoknight",
  });
});

// — Start —————
app.listen(PORT, () => {
  console.log(`\n👤 EchoknightAI webhook running on port ${PORT}`);
  console.log(`    Endpoint: POST http://localhost:${PORT}/webhook/echoknight\n`);
});

```